PROGRAMACIÓN II
Trabajo Práctico 3: Introducción a la Programación Orientada a Objetos

OBJETIVO GENERAL

Comprender los fundamentos de la Programación Orientada a Objetos, incluyendo clases, objetos, atributos y métodos, para estructurar programas de manera modular y reutilizable en Java.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Clases y Objetos	Modelado de entidades como Estudiante, Mascota, Libro, Gallina y NaveEspacial
Atributos y Métodos	Definición de propiedades y comportamientos para cada clase
Estado e Identidad	Cada objeto conserva su propio estado (edad, calificación, combustible, etc.)
Encapsulamiento	Uso de modificadores de acceso y getters/setters para proteger datos
Modificadores de acceso	Uso de private, public y protected para controlar visibilidad
Getters y Setters	Acceso controlado a atributos privados mediante métodos
Reutilización de código	Definición de clases reutilizables en múltiples contextos

Caso Práctico

Desarrollar en Java los siguientes ejercicios aplicando los conceptos de programación orientada a objetos:

1. **Registro de Estudiantes**

Crear una clase Estudiante con los atributos privados: nombre, apellido, curso, calificación.

- **Métodos requeridos:** Getters y Setters para cada atributo. mostrarInfo(), subirCalificacion(puntos), bajarCalificacion(puntos).

- **Reglas de negocio:** Los textos no pueden estar vacíos. La calificación siempre debe mantenerse en el rango de 0.0 a 10.0. Los métodos de subir y bajar nota deben validar que los puntos sumados o restados no rompan este límite.

- **Tarea en el Main:** Instanciar un estudiante e intentar setear datos inválidos (ej. nota mayor a 10 o textos vacíos) para probar tus validaciones. Luego, cargar datos correctos, mostrar su información, y utilizar los métodos para aumentar y disminuir calificaciones, probando también los límites lógicos.

2. **Registro de Mascotas**

Crear una clase Mascota con los atributos privados: *nombre, especie, edad*.

- **Métodos requeridos:** Getters y Setters para cada atributo. mostrarInfo(), cumplirAnios().
- **Reglas de negocio:** Validar que el nombre y la especie no estén vacíos. La edad nunca puede ser un valor negativo. El método cumplirAnios() debe incrementar la edad en 1 e informar el cambio.
- **Tarea en el Main:** Crear un objeto Mascota. Intentar asignarle una edad negativa para demostrar que tu objeto se protege. Luego, asignarle datos válidos, mostrar su información inicial, invocar cumplirAnios() un par de veces para simular el paso del tiempo y mostrar la información final.

3. **Encapsulamiento con la Clase Libro**

Crear una clase Libro con atributos privados: *titulo, autor, añoPublicacion*.

- **Métodos requeridos:** Getters y Setters para todos los atributos. mostrarInfo().
- **Reglas de negocio:** Validar que los textos no estén vacíos. El año de

publicación debe ser mayor a 0 y no puede ser un año en el futuro (ej. mayor al año actual).

- **Tarea en el Main:** Crear un libro. Intentar modificar el año con un valor inválido (ej. 2999) y mostrar en consola que el objeto rechazó el dato. Luego, modificar el año con uno válido y mostrar la información final para verificar que se cargó correctamente.

4. Gestión de Gallinas en Granja Digital

Crear una clase Gallina con los atributos privados: *idGallina, edad, huevosPuestos*.

- **Métodos requeridos:** Getters y Setters para todos los atributos. ponerHuevo(), envejecer(), mostrarEstado().
- **Reglas de negocio:** Ningún valor numérico de la gallina (edad o cantidad de huevos) puede ser negativo.
- **Tarea en el Main:** Crear **dos** objetos gallina diferentes (ej. gallinaA y gallinaB). Intentar asignar una edad negativa a una de ellas para probar la robustez. Luego, simular acciones de forma independiente (ej. hacer que la gallina A ponga 2 huevos y la B solo 1). Mostrar el estado final de ambas para demostrar que son entidades separadas y no comparten la memoria.

5. Simulación de Nave Espacial

Crear una clase NaveEspacial con los atributos privados: *nombre, combustible* y una constante interna para el tope máximo (ej. *CAPACIDAD_MAXIMA = 100*).

- **Métodos requeridos:** Getters y Setters. despegar(), avanzar(distancia), recargarCombustible(cantidad), mostrarEstado().
- **Reglas de negocio:** El combustible no puede ser menor a 0 ni superar la capacidad máxima al recargar. Asume un costo fijo para despegar (ej. 5 unidades) y un costo por distancia para avanzar (ej. 2 unidades por km). Debes validar que haya suficiente combustible antes de ejecutar la acción; si no alcanza, la acción debe cancelarse y mostrar un error.
- **Tarea en el Main:** Crear una nave con 50 unidades de combustible. Intentar avanzar una distancia que requiera más combustible del que tiene para probar la validación. Luego, realizar una recarga (intentando superar el límite para probar también ese fallo de seguridad), y finalmente avanzar correctamente. Mostrar el estado final de la nave.

CONCLUSIONES ESPERADAS

- Comprender la diferencia entre clases y objetos.
- Aplicar principios de encapsulamiento para proteger los datos.
- Usar getters y setters para gestionar atributos privados.
- Implementar métodos que definen comportamientos de los objetos.
- Manejar el estado y la identidad de los objetos correctamente.
- Aplicar buenas prácticas en la estructuración del código orientado a objetos.
- Reforzar el pensamiento modular y la reutilización del código en Java.